

LexDraft

Application Architecture

Front end · back end · data · deployment · security

Audience	Engineers, technical PMs, reviewers onboarding to LexDraft
Status	Living document · mirrors current monorepo state
Repo	apps/web (Vite + React 18) · apps/api (Node + Express) · packages/*
Generated	2026-05-26

"LexDraft is a multi-tenant practice-management SaaS for Indian advocates — drafting, review, matter management, cause-list, research, billing, client portal, and a SuperAdmin control plane — sold in Solo / Practice / Firm tiers."

Contents

1	What LexDraft is
2	High-level architecture
3	Technology stack
4	Repository layout
5	Frontend architecture (apps/web)
6	Backend architecture (apps/api)
7	Request lifecycle · auth / plan / tenant / RBAC
8	Data model & persistence
9	Plans, roles & feature gating
10	AI integration · Anthropic Claude
11	Client portal sub-app
12	SuperAdmin control plane
13	Deployment, observability & security
14	End-to-end workflows
15	Reference index

What LexDraft is

LexDraft is a **multi-tenant SaaS** built for Indian advocates and law firms. It packages the day-to-day practice into a single web application: drafting, contract review, mock arguments, matter / case management, cause-list with eCourts hooks, legal research, billing, a separate client portal, and firm administration.

Core surfaces

Drafting	Template + Claude-assisted document generation across Indian languages.
Contract review	Clause-by-clause AI review with reviewer workflow + comment threads.
Mock arguments	Opposing-counsel rebuttal generator with polish / improve stages.
Matter management	Notes, tasks, documents, hearings, limitation deadline tracking.
Cause-list / hearings	Today + upcoming view; eCourts CNR sync seam.
Research	Lex.AI canned answer + semantic statute search (vector embeddings).
Billing	Invoices, expenses, financial exports.
Client portal	Separate sub-app with magic-link auth and scoped JWT.
Firm admin	Members, roles, analytics, audit log, MFA, DPDP self-service.

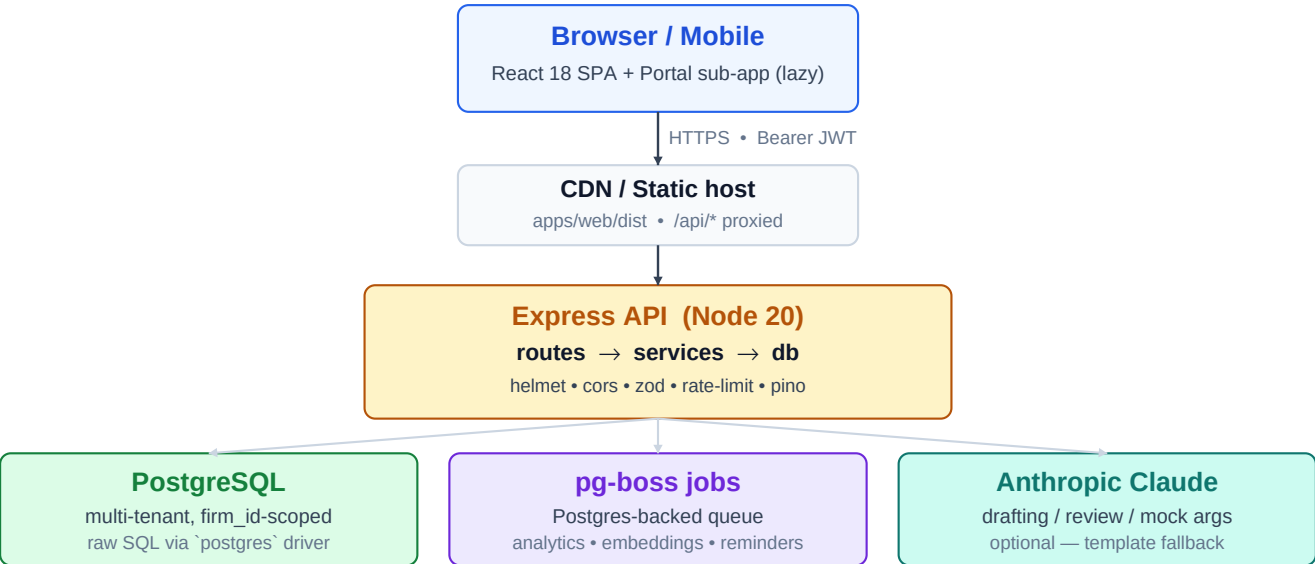
Plans

Solo (1 seat · 20 AI drafts/mo) · **Practice** (2–8 seats · 200 AI drafts/mo) · **Firm** (9+ seats · 1000 AI drafts/mo).
An internal-only **SuperAdmin** control plane is operated by LexDraft staff.

High-level architecture

The system is a classic single-page-app + REST API + Postgres stack with a job queue and an optional LLM dependency. Every protected query is filtered by **firm_id**; the API itself is **stateless** (bearer-JWT, no server session), so it scales horizontally by replicating the Node process behind any load balancer.

End-to-end request path



Browser → static host → Express API → Postgres / pg-boss / Claude. Webhooks land at /api/webhooks/:source with per-provider HMAC verification.

Non-functional properties

Tenant isolation	Every protected service helper takes firm_id from req.user and filters every query. No cross-tenant reads in v1.
Statelessness	JWT bearer only — no server-side session store. Horizontal scale by replicating the Node process.
Background work	pg-boss queue handles analytics refresh, embedding generation, scheduled reminders.
Search	Postgres + vector embeddings for laws/research semantic search; garble filter screens OCR noise before LLM calls.
Multi-origin	Single-origin in dev (Vite proxy); CORS-allowlisted multi-origin in prod.

Technology stack

Runtime & tooling

Tool	Role
Node.js	≥ 20 (.nvmrc pins 20.18.0)
TypeScript	5.6 · strict mode across every workspace
pnpm	≥ 9 (workspace package manager)
Turborepo	2 (orchestrates per-workspace tasks)
Vite	5 (web build + dev server with /api proxy)

Backend (apps/api)

Express 4 + Zod	HTTP routing with Zod input validation on every route.
postgres (npm)	SQL driver; raw SQL via tagged templates — no ORM, deliberate.
pg-boss	Postgres-backed background job queue.
jsonwebtoken + bcryptjs	HS256 JWT bearer auth; bcrypt password hashing.
otplib + qrcode	TOTP MFA enrolment + verification.
mammoth + pdf-parse	Server-side text extraction from DOCX / PDF uploads.
pino + pino-pretty	Structured logging with requestId on every error path.
helmet / cors / compression / morgan / express-rate-limit	HTTP hardening.
Anthropic SDK	Optional — Claude calls when ANTHROPIC_API_KEY is set.

Frontend (apps/web)

React 18 + react-router-dom 6	SPA with nested routes, code-split portal sub-app.
@tanstack/react-query 5	One hook per backend resource in apps/web/src/hooks/.
Zustand 5	Cross-cutting UI / auth state — current user, JWT, theme, density, language.
axios	Interceptor attaches JWT and clears the session on 401 (lib/api.ts).
framer-motion + lenis	Transitions + smooth scroll.
mammoth + pdfjs-dist	Client-side document parsing / preview.
html2canvas + jsPDF	Client-side PDF export from rendered DOM.
Tailwind CSS 3	Workspace preset bound to design tokens in styles/tokens.css.

SECTION 4

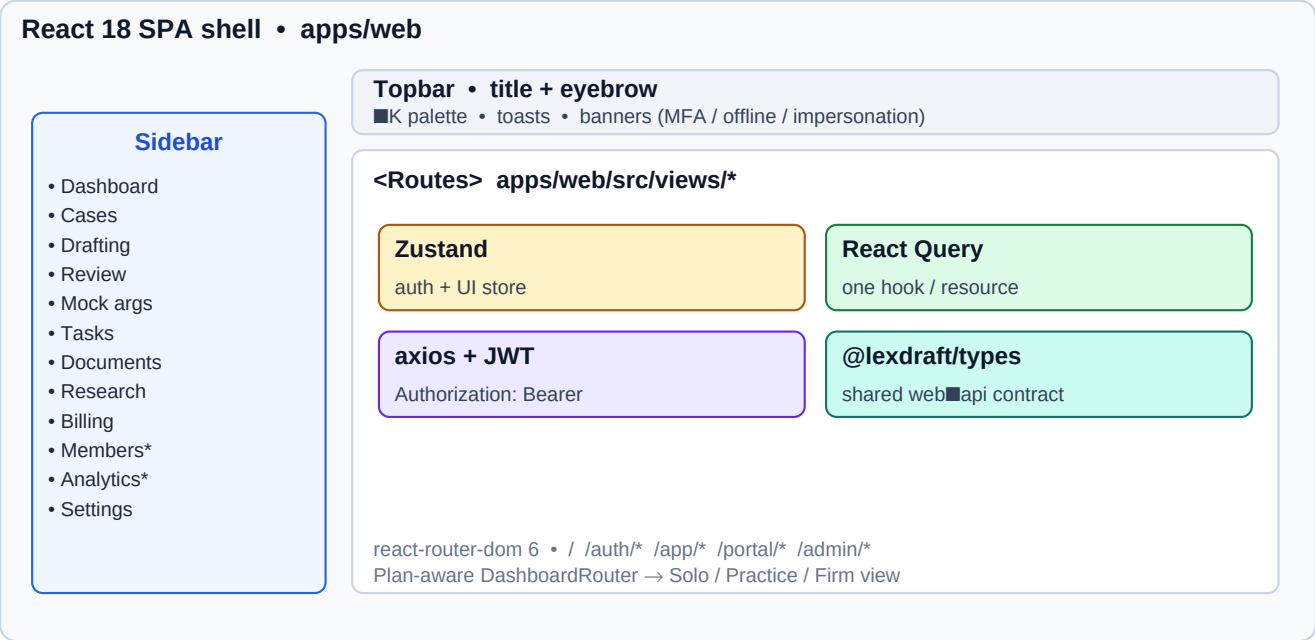
Repository layout

A pnpm workspace orchestrated by Turborepo. Two deployable apps (apps/web, apps/api) consume a small set of shared packages so the web/api contract stays type-safe in a single edit.



Frontend architecture

The web app is a single-page React 18 application served from apps/web/dist. It owns the entire visual surface; everything else is fetched from the API behind a Bearer JWT. The router is plan-aware — the same /app/dashboard URL renders a different view per user plan via DashboardRouter.



Shell composition. Sidebar entries marked * are visible only on Practice / Firm plans.

Routing model



State management

Zustand (auth)	Current user, JWT, impersonation actAs, MFA challenge state. Persisted to localStorage; refreshed on boot via refreshUser().
Zustand (ui)	Theme, density, language, command-palette open state, toast queue.
React Query	All server state. One hook per resource — useCases, useCaseNotes, useTasks, useMockArguments, useEcourts, ...
axios	Single client in lib/api.ts with interceptors that attach Authorization: Bearer and clear the session on 401.
localStorage	Theme, density, language, JWT. Read once on boot; written on change.

Design system

Tokens live in apps/web/src/styles/tokens.css as CSS custom properties — a 1:1 mirror of design-system.md. The Tailwind preset in @lexdraft/tailwind-config reads the same tokens so utility classes and component CSS stay in sync. Dark theme is default; light theme is a single <html data-theme="light"> swap.

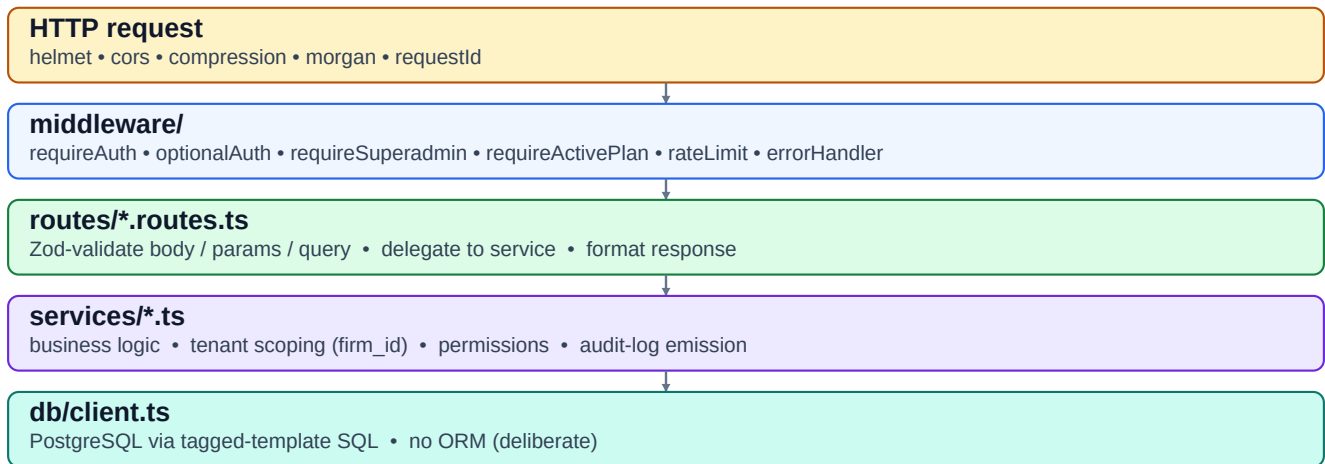
§2 → tokens	--bg-base, --text-primary, status colors.
§3 → typography	.display-xl, .heading-md, .body-md, .eyebrow, .mono.
§4 → spacing	--space-0...--space-13.
§5 → radii	--radius-sm / md / lg / xl / full.
§7 → components	.btn, .input, .card, .chip, .badge, .tbl, .pill-nav, .stat-row.
§8 → layout	.app, .sidebar, .topbar, .content, .mobile-nav.

Adding a new view

1. Create apps/web/src/views/MyView.tsx. 2. Register the route in App.tsx. 3. Add the sidebar entry in components/shell/nav-config.ts. 4. Add the topbar title + eyebrow in the same file's ROUTE_TITLES map.

Backend architecture

The API is a layered Express application. The contract is unambiguous: **routes never touch the database**. Routes Zod-validate the request and delegate; services own the business logic and the only legal callers of db(); the database layer exposes Postgres via tagged-template SQL, deliberately without an ORM.



Backend layering. The boundary between routes and services is enforced by convention and by every PR review.

Route map (apps/api/src/routes/index.ts)

requireActivePlan returns **402 PaymentRequired** when firms.plan_status is past_due / cancelled, or when renews_at is in the past for a non-trial firm. SuperAdmins + impersonation sessions bypass the check. /me* is deliberately excluded so users can still log in to manage their profile / billing after a plan lapses.

Mount	Auth	Plan gate	Purpose
GET /api/health	public	—	Liveness — process is up
GET /api/ready	public	—	Readiness — DB ping included
/api/auth/*	public	—	Sign-in, sign-up, sign-out, me
/api/webhooks/:src	upstream HMAC	—	Inbound eCourts / payments / e-sign
/api/survey/*	public, rate-limited	—	Market-research questionnaire
/api/portal/*	portal JWT	—	Client portal — magic-link auth
/api/invitations/*	mixed	—	Accept-invite + admin management
/api/dashboard	requireAuth	plan	Aggregated home payload (Solo)
/api/firm/*	requireAuth	plan	Firm-scoped dashboard / members / settings
/api/cases, /api/case-notes	requireAuth	plan	Matters CRUD + per-matter notes
/api/hearings, /api/diary, /api/causelist	requireAuth	plan	Hearings / diary / cause list

Mount	Auth	Plan gate	Purpose
/api/tasks	requireAuth	plan	Kanban TaskBoard
/api/documents, /api/drafts	requireAuth	plan	Document register + persisted drafts
/api/drafting	requireAuth	plan	POST /generate — Claude or template
/api/review	requireAuth	plan	Contract review workflow + comments
/api/mock-arguments	requireAuth	plan	Mock arguments + polish + improvements
/api/title-reports	requireAuth	plan	Title Investigation Report wizard
/api/matter-intel, /api/matter-chat	requireAuth	plan	Matter intelligence + chat
/api/research, /api/laws	requireAuth	plan	Lex.AI answer + semantic statute search
/api/ecourts	requireAuth	plan	eCourts CNR sync
/api/clients, /api/leads	requireAuth	plan	CRM
/api/invoices, /api/expenses, /api/exports	requireAuth	plan	Billing
/api/limitations, /api/sanhita, /api/calculators	requireAuth	plan	Statute + calculators
/api/conflicts, /api/coverage	requireAuth	plan	Conflict-check + cover-counsel
/api/practice-analytics, /api/analytics, /api/caseload-health	requireAuth	plan	Analytics & diagnostics
/api/engagement, /api/letterheads, /api/clauses	requireAuth	plan	Firm templates + clause library
/api/archive, /api/physical-documents	requireAuth	plan	Closed matters + physical-file register
/api/me	requireAuth	—	Profile + resolved feature set
/api/me/mfa	optionalAuth	—	TOTP enrolment + challenge step
/api/me/dpdp	requireAuth	—	DPDP self-service (export / delete)
/api/portal-admin	requireAuth	plan	Firm-side portal administration
/api/admin/*	requireAuth + superadmin	—	Platform admin (impersonation blocked here)
/api/admin/errors	requireAuth + superadmin	—	Internal error log viewer

Service inventory

Each file in `apps/api/src/services/` owns one business capability. The whole list is large; the highlights below trace the major flows referenced elsewhere in this document.

<code>auth.service.ts</code>	Sign-in / sign-up, JWT issuance, password hashing, demo-account provisioning.
<code>permissions.service.ts</code>	Three-layer feature gate — $\text{Baseline} \vee (\text{Plan} \wedge \text{Role} \wedge \neg \text{deny} \vee \text{grant})$.
<code>cases.service.ts</code> · <code>case-notes.service.ts</code>	Matter CRUD + per-matter timeline.
<code>hearings.service.ts</code> · <code>diary.service.ts</code>	Today / upcoming hearings; full diary view.
<code>tasks.service.ts</code>	Kanban states (pending → progress → review → done).
<code>documents.service.ts</code> · <code>drafts.service.ts</code>	Document register + persisted draft output.
<code>drafting.service.ts</code>	Claude-assisted generation with deterministic template fallback.
<code>review.service.ts</code> · <code>review-comments.service.ts</code>	Contract review + comment threads.
<code>mock-arguments.service.ts</code>	Opposing-counsel rebuttal — polish + improvements pipeline.
<code>title-reports.service.ts</code>	Title Investigation Report wizard with enforced state machine.
<code>research.service.ts</code> · <code>laws-search.service.ts</code> · <code>embeddings.service.ts</code>	Research answer + semantic statute search; garble filter for OCR noise.
<code>ecourts.service.ts</code> · <code>case-sync.service.ts</code>	eCourts CNR sync.
<code>matter-intel.service.ts</code> · <code>matter-chat.service.ts</code>	Matter intelligence + chat.
<code>clients.service.ts</code> · <code>leads.service.ts</code>	Client + intake CRM.
<code>invoices.service.ts</code> · <code>expenses.service.ts</code> · <code>exports.service.ts</code>	Billing pipeline.
<code>limitations.service.ts</code> · <code>limitations.calculator.ts</code>	Limitation Act deadline math.
<code>analytics.service.ts</code> · <code>practice-analytics.service.ts</code> · <code>analytics-refresh.service.ts</code>	Firm + practice analytics, materialized refresh.
<code>caseload-health.service.ts</code>	Stale-matter / overdue diagnostics.
<code>conflicts.service.ts</code> · <code>coverage.service.ts</code>	Conflict-checks + cover-counsel requests.
<code>sanhita.service.ts</code> · <code>calculators.service.ts</code>	Statute reference (BNS / BNSS / BSA) + stamp / court-fee / limitation calculators.
<code>engagement.service.ts</code> · <code>templates.service.ts</code> · <code>letterheads.service.ts</code> · <code>clauses.service.ts</code>	Firm templates + clause library.
<code>firm.service.ts</code> · <code>firm-admin.service.ts</code> · <code>firm-enquiries.service.ts</code>	Firm CRUD + admin actions + sales enquiries.

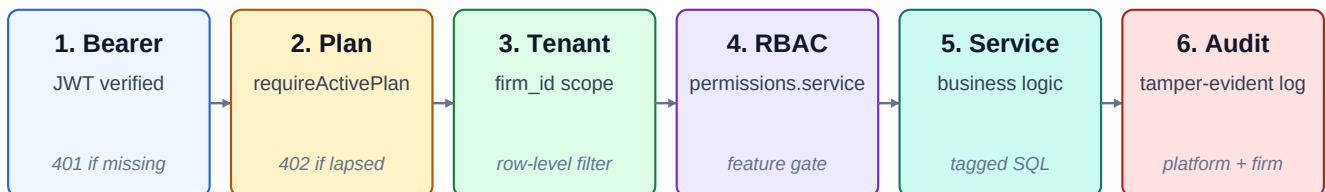
invitations.service.ts · mfa.service.ts · dpdp.service.ts	Invites, TOTP MFA, DPDP self-service.
audit.service.ts · impersonation.service.ts	Tamper-evident audit log + SuperAdmin impersonation with banner.
notifications.service.ts · cache-broadcaster.ts · storage.service.ts	Notifications, cache invalidation, blob storage.
plan-status.service.ts · portal-plan-gate.ts	Subscription state evaluation.
portal.service.ts	Client portal — magic links, scoped JWT, matter visibility.
survey.service.ts · survey-draft.service.ts	Public market-research questionnaire.
admin.service.ts · error-log.service.ts	Platform admin + internal error viewer.
jobs.service.ts	pg-boss queue — analytics refresh, embeddings, reminders.
tenant.ts	Tenant scoping helpers used by every service.

Middleware

requireAuth	Verifies JWT, attaches req.user. Rejects with 401.
optionalAuth	Verifies JWT if present. Used for the MFA challenge step (post-password, pre-bearer).
requireSuperadmin	Gates /api/admin/*. Blocks impersonation sessions.
requireActivePlan	Returns 402 when the firm's plan has lapsed. SuperAdmins + impersonation bypass.
rateLimit	IP-level limits with tighter signUpLimiter and looser surveyDraftLimiter.
errorHandler	Zod failures → 400; unknown errors → 500; attaches requestId for tracing.

Request lifecycle

A single authenticated request runs a deterministic gate chain. Each gate has its own status code so a client can react precisely — a 401 means re-auth, a 402 means renew the plan, a 403 means the role lacks the feature.



Request fails fast — each gate has its own status code so the client can react precisely.

Authentication

1. POST /api/auth/sign-in with { email, password }. 2. The API returns a 7-day **HS256 JWT** (JWT_EXPIRES_IN configurable). 3. The web client persists the JWT in Zustand + localStorage. 4. The axios interceptor in lib/api.ts attaches Authorization: Bearer <jwt> on every /api/* call and clears the session on 401. 5. GET /api/auth/me returns the user + resolved feature set. 6. Sign-out is stateless — the client just discards the token.

MFA (TOTP)

Enrolment: scan QR (otppath URL) → enter 6-digit code → backend stores HMAC-protected secret + recovery codes. Sign-in: if MFA enabled, password step returns a challengeId; /api/me/mfa/verify-challenge exchanges code + challenge for the bearer. That endpoint uses optionalAuth because the user does not yet hold a bearer.

Tenant isolation

Every protected service helper takes the resolved firm_id from req.user and filters every query. Cross-tenant reads are not permitted in v1. SuperAdmin endpoints under /api/admin/* bypass tenant scoping by design; impersonation routes attach actAs.firm_id with a persistent UI banner and audit trail.

Data model & persistence

Schema lives in numbered SQL migrations under `apps/api/migrations/`. The runner is `apps/api/src/scripts/migrate.ts`. Production uses PostgreSQL; the driver is the `postgres` npm package with raw SQL through tagged templates. Every protected query is filtered by `firm_id` via helpers in `services/tenant.ts`.

Core entities

firms	Tenant. <code>plan_tier</code> (Solo / Practice / Firm), <code>plan_status</code> (trial / active / past_due / cancelled), <code>renews_at</code> , <code>billing</code> .
users	Belongs to a firm. Role + <code>is_superuser</code> flag.
cases · case_notes	Matters with per-matter timeline; <code>case_type</code> , stage, court fields, parties, <code>client_id</code> .
hearings	Listed dates, court, status; ingested via webhook from eCourts where wired.
tasks	Kanban states pending → progress → review → done.
documents · drafts	Document register + persisted Claude / template output.
clients · leads	CRM.
invoices · expenses	Billing.
limitations	Statutory deadline tracker, computed by <code>limitations.calculator.ts</code> .
diary · archive · physical_documents	Operational records.
clauses · letterheads · engagement_templates	Firm-branded templates.
mock_arguments + polish / review_raw / improvements / language	Mock-arguments pipeline.
reviews + review_comments	Contract-review workflow.
title_report_* (13 tables)	TIR header + property + chain + documents + encumbrances + searches + litigation + approvals + heirs + defects + <code>ai_runs</code> + exports + per-firm-year counter.
portal_accounts · portal_messages · portal_acks	Client portal.
invitations	Single-use 24h tokens.
mfa_secrets · mfa_recovery_codes	TOTP enrolment.
dmdp_*	DPDP exports / erasure requests / consent ledger.
audit_log	Tamper-evident, indefinitely retained.
error_log	Internal-only viewer at <code>/api/admin/errors</code> .

analytics_*	Materialized views refreshed by background job.
embeddings	Vector embeddings for laws / research semantic search.

Migration spine

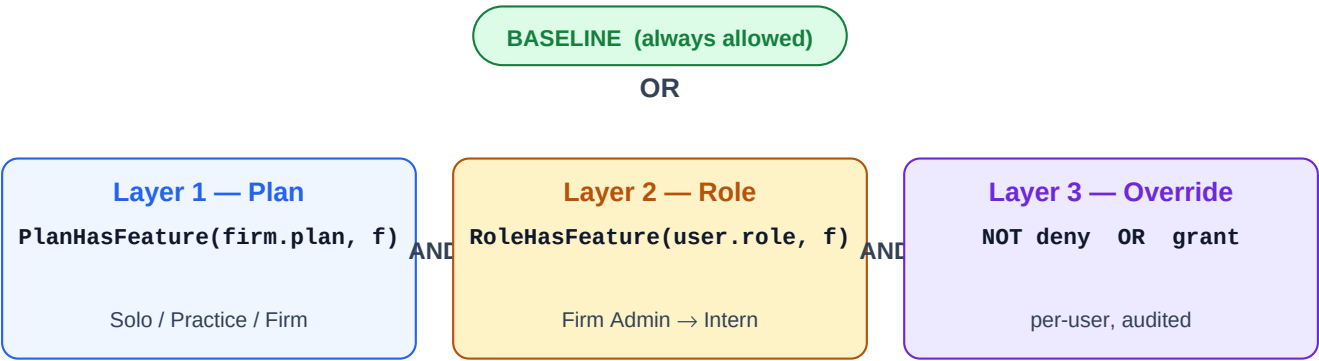
Migrations are append-only and numbered. Schema evolves forward; rollbacks are handled by writing a follow-up migration, never by editing a prior file. Significant milestones:

0001 → 0008	Core tables + clauses + demo data + initial RBAC scaffolding.
0009 → 0016	RBAC core, client portal, portal messages, AI-draft quota for Solo.
0017 → 0024	Physical-files, index tuning, MFA, DPDP, materialized analytics, limitation reference.
0025 → 0030	Error log, contract reviews, review workflow, letterheads, sign-up profile fields.
0031 → 0039	Survey + drafts, case notes, firm enquiries, mock arguments pipeline (base → polish → review-raw → improvements → language).
0040 →	Plan AI caps, eCourts phase 2, title reports (0050+), and ongoing additions.

Plans, roles & feature gating

Plan determines surface area; role determines actions within that surface. Solo users cannot see Members or Analytics in the nav; deep-links are redirected to /app/dashboard. GET /api/me returns the resolved feature set so the web client can render the right sidebar / buttons without re-computing the rule.

Three-layer feature gate (apps/api/src/services/permissions.service.ts)



Plans

Plan	Surface
Solo	1 seat. Solo dashboard. AI-draft quota 20/mo. No Firm sidebar group.
Practice	2–8 seats. Practice dashboard (chambers pulse + members table). AI-draft quota 200/mo. Members + Settings in Firm group; no Analytics.
Firm	9+ seats. Full Firm dashboard (KPIs, revenue, members, practice mix, top clients). AI-draft quota 1000/mo. Full Firm sidebar including Analytics.

Tenant roles

Firm Admin	Full feature access + user management + billing. Bootstrap user when a tenant is provisioned.
Practice Group Lead	All drafting features + manage users within their practice group.
Partner	Full drafting, review, e-sign, matter management, billing view.
Senior Associate	Drafting, AI drafting, clause library, review, e-sign.
Associate	Drafting, AI drafting, clause library.
Paralegal	Limited drafting, templates, document assembly.
Legal Secretary	Document formatting, calendar, basic templates.
Intern / Trainee	Read-only or restricted drafting.
External Client (v2)	View-only via the client portal.

Custom	Firm Admins can clone/extend any system role; tenant-scoped.
--------	--

Invariants: every user has exactly one role · a tenant must always have ≥ 1 active Firm Admin (last-admin demotion blocked at the API) · role changes write to the tenant audit log.

AI integration

LexDraft calls Claude for drafting, contract review, mock arguments, research answers and title-report defects/opinion analysis. Embeddings power the semantic statute search. The integration is designed to be **optional in development** — every AI surface has a deterministic template fallback, so the entire UI is usable without an API key.

Feature	Service	Model env
Drafting	drafting.service.ts	ANTHROPIC_MODEL (default claude-sonnet-4-6)
Contract review	review.service.ts	Same
Mock arguments + polish + improvements	mock-arguments.service.ts	Same
Title reports — defects + opinion	title-reports.service.ts	Same (with xAI fallback path)
Research answer	research.service.ts	Same
Laws semantic search	laws-search.service.ts + embeddings.service.ts	Embedding model + retrieval

Design rules

- ANTHROPIC_API_KEY is optional — services return deterministic templates when unset.
- **Structured briefs** — each Claude call is composed from typed schemas in apps/web/src/lib/doc-schemas.ts and mirror types in @lexdraft/types.
- **Per-user quotas** — AI drafts are capped per user, per firm billing cycle: Solo 20 · Practice 200 · Firm 1000 (plan_ai_caps). Counts are append-only; deleting a draft tombstones the linked ai_generations row but never refunds quota.
- **Garble / quality filters** — laws-search.service.ts filters OCR-noise chunks before sending them to the LLM.
- **Eval harness** — apps/api/eval/runner.ts (pnpm --filter @lexdraft/api eval) runs offline regression suites against AI features.

SECTION 11

Client portal sub-app

apps/web/src/views/portal/ is a separate, lazy-loaded SPA with its own auth and a stripped-down surface. Portal sessions cannot reach /api/admin/* or any non-/api/portal* route.

Auth	Magic-link → exchange at /api/portal/auth/* for a scoped portal JWT with a different audience than the firm-side JWT.
Views	PortalLoginView · PortalDashboardView · PortalMatterDetailView · PortalMessagesView · PortalProfileView.
Visibility	Which matters / documents a client sees is controlled by toggles persisted via migrations 0015 + 0014.
Firm-side admin	PortalInboxView + /api/portal-admin/* give the firm an inbox to manage portal threads.
Isolation	Portal sessions are blocked from any firm-side route. The sub-app is lazy-loaded so it does not bloat the firm-side bundle.

SECTION 12

SuperAdmin control plane

apps/web/src/admin/ renders inside AdminShell at /admin/*. The control plane is the operational surface for LexDraft staff: tenant lifecycle, identity & impersonation, billing operations, customer-health, support tooling, DPDP / compliance and platform operations.

AdminDashboardView	Platform KPIs.
FirmsView / FirmDetailView	Provision, suspend, view a tenant.
UsersView	Cross-tenant user search.
AuditLogView	Tamper-evident log.
TemplatesView	Platform-managed templates.
ErrorLogView	Internal error log (/api/admin/errors).
ImpersonationBanner	Persistent banner whenever an admin is acting as a tenant user; impersonation is blocked from /api/admin/*.

SuperAdmin roles

Founder / Owner	Impersonate ✓ · Billing ✓ · Compliance ✓
Support engineer	Impersonate ✓ (with reason) · Compliance read-only
Billing operations	Billing ✓
Compliance officer	Compliance ✓ + edit
Read-only auditor	Read across compliance views

Deployment, observability & security

Build & deploy

pnpm build produces apps/web/dist/ (static SPA) and apps/api/dist/ (Node bundle). A reasonable deploy:

1. Build the API container with node:20-alpine, copy apps/api/dist + workspace node_modules, set env vars, expose PORT.
2. Build the web app and serve from any CDN (CloudFront / Vercel / S3 + CloudFront / Netlify). Set VITE_API_URL at build time.
3. Route the CDN's /api/* path to the API, or hard-code the API origin and configure the API's CORS_ORIGINS accordingly.

docker-compose.yml / docker-compose.prod.yml ship local + production-shaped stacks (db + api + web). Health probes:

- GET /api/health — cheap liveness, no DB.
- GET /api/ready — readiness, pings DB, returns 503 when dependencies are down.

Observability & operations

Logging	pino structured JSON; pino-pretty in dev. Every error path attaches a requestId.
Error log	error-log.service.ts records persistent errors; /api/admin/errors exposes a SuperAdmin viewer.
Audit log	audit.service.ts writes tamper-evident events for tenant + platform actions. Indefinite retention.
Background jobs	pg-boss queue via jobs.service.ts. Workers run analytics refresh, embedding generation, reminders.
Cache invalidation	cache-broadcaster.ts fans out invalidations across processes.
Rate limiting	express-rate-limit with per-route tuning (signUpLimiter, surveyDraftLimiter).
Health probes	/api/health (liveness) · /api/ready (readiness with DB ping).

Security model

JWT (HS256)	Configurable secret + expiry. Bearer-only; no cookies.
Password hashing	bcryptjs with sensible cost.
MFA	TOTP via otplib, QR via qrcode. Recovery codes per user.
Tenant isolation	Every protected query filtered by firm_id; no cross-tenant reads in v1.
Plan gate	requireActivePlan returns 402 when a firm's subscription has lapsed (excluding /me*).
Hardening	helmet, strict cors allowlist, compression, morgan, rate-limits, Zod validation on every route.
Webhooks	Per-provider HMAC via webhooks.verify.ts.
DPDP	Export + erasure + consent ledger via dpdp.service.ts and /api/me/dpdp.

Impersonation	Allowed only off /api/admin/*; persistent UI banner; reason captured and audited.
Audit log	Tamper-evident, indefinitely retained, queryable in AuditLogView.

End-to-end workflows

Tenant provisioning



Drafting



Contract review



Mock arguments

1. User submits the case posture in MockArgumentsView (party, facts, contentions, language).
2. mock-arguments.service.ts generates an opposing-counsel rebuttal via Claude.
3. **Polish stage** refines tone and citation density.
4. **Improvements stage** reviews the raw output stored in mock_arguments_review_raw.
5. Multi-language — supported languages enumerated in apps/api/src/lib/languages.ts.

Title reports (TIR)

Wizard-driven, AI-assisted, ends in a letterhead PDF. The state machine is enforced by the API in `title-reports.service.ts`, not just the UI.



- **draft** → **in_review**: requires property + ≥ 1 chain link + ≥ 1 EC row + ≥ 1 search + a non-pending verdict.
- **in_review** → **finalised**: requires every blocker defect acknowledged/dismissed + an opinion summary.
- **finalised** → **issued**: requires a PDF export to have been generated.
- AI runs persisted in `title_report_ai_runs` for replay/audit; deterministic template fallback means the feature works end-to-end without `ANTHROPIC_API_KEY`.
- Solo plan capped at 2 reports per cycle (`plan_title_report_caps`); cap enforced before sequence-number allocation so a 429 doesn't burn a number.

Cause-list / hearings

eCourts CNR sync (where wired) feeds `hearings.service.ts`. The dashboard pulls `GET /api/hearings/today`; the diary view pulls the broader window. Limitation calculations live in `limitations.calculator.ts` and surface as the limitation index on every dashboard.

Research / laws-search

`ResearchView` posts a question to `/api/research?q=`; `research.service.ts` returns a canned Lex.AI answer + citations. `LawSidePanel` calls `/api/laws` for semantic statute lookup. The service applies a **garble filter** to suppress OCR-garbage chunks, distinguishes **central vs state acts**, and supports per-state filtering.

Client portal

External clients receive a magic link → exchange for a portal JWT at `/api/portal/auth/*` → land on the stripped-down portal app under `apps/web/src/views/portal/`. Portal sessions cannot reach firm-side routes; the sub-app is lazy-loaded so it does not bloat the firm-side bundle.

Reference index

Where to look next — canonical sources of truth and the documents that go deeper than this overview can.

API contract — source of truth	packages/types/src/index.ts
Backend route map	apps/api/src/routes/index.ts
Permissions / feature gate	apps/api/src/services/permissions.service.ts
Tenant scoping helpers	apps/api/src/services/tenant.ts
Web routing entry	apps/web/src/App.tsx
Auth store + axios client	apps/web/src/store/auth.ts · apps/web/src/lib/api.ts
Design tokens	apps/web/src/styles/tokens.css · design-system.md
Plans, pricing, gating	LexDraft_Pricing_and_Tiers.docx
Per-plan dashboard model	WORKFLOW_DASHBOARDS.md
RBAC + tenant onboarding spec	lexdraft-user-management-spec.md
Client portal spec	CLIENT_PORTAL.md
Deployment runbook	DEPLOYMENT.md
Title reports primer + prompts	TITLE_REPORTS.md
Matter intelligence primer	MATTER_INTELLIGENCE.md
Feature inventory	LEXDRAFT_FEATURES_ANALYSIS.md
Onboarding	ONBOARDING.md
Privacy / Terms / DPA	PRIVACY_POLICY.md · TERMS_OF_SERVICE.md · DATA_PROCESSING_AGREEMENT.md
Roadmap	LEXDRAFT_ROADMAP.md

Status

This document is a living architectural reference for LexDraft and mirrors the current monorepo state. Where this document and code diverge, code wins — consider that a bug in the document and open a small PR.